



DOCUMENTO 06

Base de Datos MySQL

Script de creación, restricciones, datos de prueba y consultas principales

Proyecto	PARQUEADERO — Sistema de gestión de parqueadero
Institución	[Pendiente de completar]
Integrantes	[Pendiente de completar]
Versión / Fecha	1.0 — 1 de octubre de 2026

Base de Datos MySQL

Alcance de este documento y fuentes de información

Este documento distingue tres tipos de información:

- **EXISTENTE** Elementos verificados en el repositorio del proyecto a la fecha de elaboración.
- **PROPUESTO** Diseño planteado para el sistema de parqueadero, aún no implementado en el código.
- **PENDIENTE** Datos que deben ser completados por el equipo del proyecto.

El video de referencia corresponde a una sesión de clase que muestra la **metodología y las herramientas** de desarrollo (repositorio en GitHub, frontend PWA con Vite, base de datos MySQL en Clever Cloud y despliegue en Vercel). El video **no muestra funcionalidades del sistema de parqueadero**; por esta razón, las funcionalidades descritas se presentan como diseño propuesto.

Estado de la base de datos

El repositorio **no contiene todavía scripts SQL ni consultas** en el código. El siguiente script es la implementación en MySQL del modelo entidad-relación propuesto (documento 05). Los signos `?` representan parámetros que el backend debe enviar mediante consultas preparadas, para prevenir inyección SQL.

1. Creación de la base de datos y tablas

```
-- =====
-- PARQUEADERO - Systems & Programación
-- Script MySQL 8 (DISEÑO PROPUESTO)
-- =====
CREATE DATABASE IF NOT EXISTS parqueadero_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE parqueadero_db;

CREATE TABLE roles (
  id_rol      INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(30)  NOT NULL UNIQUE,
  descripcion VARCHAR(150) NULL
) ENGINE=InnoDB;

CREATE TABLE usuarios (
  id_usuario  INT AUTO_INCREMENT PRIMARY KEY,
  id_rol      INT            NOT NULL,
  documento   VARCHAR(20)   NOT NULL UNIQUE,
  nombres     VARCHAR(80)   NOT NULL,
  apellidos   VARCHAR(80)   NOT NULL,
```

```

    correo          VARCHAR(120) NOT NULL UNIQUE,
    telefono        VARCHAR(20)  NULL,
    tipo_usuario    ENUM('ESTUDIANTE','PROFESOR','PERSONAL') NOT NULL,
    contrasena_hash VARCHAR(255) NOT NULL,
    estado          ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    fecha_creacion  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_usuarios_rol FOREIGN KEY (id_rol)
        REFERENCES roles(id_rol) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE tipos_vehiculo (
    id_tipo_vehiculo INT AUTO_INCREMENT PRIMARY KEY,
    nombre           VARCHAR(30) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE vehiculos (
    id_vehiculo      INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario       INT          NOT NULL,
    id_tipo_vehiculo INT          NOT NULL,
    placa            VARCHAR(10) NOT NULL UNIQUE,
    marca            VARCHAR(40) NULL,
    modelo           VARCHAR(40) NULL,
    color            VARCHAR(30) NULL,
    estado           ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    fecha_registro   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vehiculos_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuarios(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_vehiculos_tipo FOREIGN KEY (id_tipo_vehiculo)
        REFERENCES tipos_vehiculo(id_tipo_vehiculo) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE zonas (
    id_zona          INT AUTO_INCREMENT PRIMARY KEY,
    nombre           VARCHAR(50) NOT NULL UNIQUE,
    descripcion       VARCHAR(150) NULL
) ENGINE=InnoDB;

CREATE TABLE espacios (
    id_espacio       INT AUTO_INCREMENT PRIMARY KEY,
    id_zona           INT          NOT NULL,
    id_tipo_vehiculo INT          NOT NULL,
    codigo            VARCHAR(10) NOT NULL UNIQUE,
    estado            ENUM('DISPONIBLE','OCUPADO','INHABILITADO') NOT NULL DEFAULT 'DISPONIBLE',
    CONSTRAINT fk_espacios_zona FOREIGN KEY (id_zona)
        REFERENCES zonas(id_zona) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_espacios_tipo FOREIGN KEY (id_tipo_vehiculo)
        REFERENCES tipos_vehiculo(id_tipo_vehiculo) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE registros (
    id_registro       INT AUTO_INCREMENT PRIMARY KEY,
    id_vehiculo        INT          NOT NULL,
    id_espacio         INT          NOT NULL,
    id_vigilante_ingreso INT        NOT NULL,
    id_vigilante_salida INT         NULL,
    fecha_hora_ingreso DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_hora_salida  DATETIME NULL,

```

```

observaciones          VARCHAR(255) NULL,
CONSTRAINT fk_registros_vehiculo FOREIGN KEY (id_vehiculo)
REFERENCES vehiculos(id_vehiculo) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_registros_espacio FOREIGN KEY (id_espacio)
REFERENCES espacios(id_espacio) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_registros_vig_ingreso FOREIGN KEY (id_vigilante_ingreso)
REFERENCES usuarios(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_registros_vig_salida FOREIGN KEY (id_vigilante_salida)
REFERENCES usuarios(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_registros_fechas
CHECK (fecha_hora_salida IS NULL OR fecha_hora_salida >= fecha_hora_ingreso)
) ENGINE=InnoDB;

CREATE INDEX idx_registros_abiertos ON registros (id_vehiculo, fecha_hora_salida);
CREATE INDEX idx_registros_ingreso ON registros (fecha_hora_ingreso);

```

2. Resumen de restricciones

Tipo	Aplicación
PRIMARY KEY	Todas las tablas usan una llave primaria entera AUTO_INCREMENT.
FOREIGN KEY	9 llaves foráneas con ON DELETE RESTRICT para impedir borrar datos referenciados.
UNIQUE	roles.nombre, usuarios.documento, usuarios.correo, tipos_vehiculo.nombre, vehiculos.placa, zonas.nombre, espacios.codigo.
ENUM	tipo_usuario, estados de usuario, vehículo y espacio.
CHECK	La fecha de salida no puede ser anterior a la de ingreso.
DEFAULT	Estados iniciales y fechas de creación automáticas.

3. Datos de prueba

Aclaración

El proyecto no tiene datos reales. Los siguientes registros son **ficticios** y solo sirven para probar el sistema.

```

-- -----
-- Datos de prueba (FICTICIOS, solo para pruebas)
-- contraseña_hash: reemplazar por hashes bcrypt generados por el backend
-- -----
INSERT INTO roles (nombre, descripcion) VALUES
('ADMINISTRADOR', 'Gestiona usuarios, vehículos, zonas y espacios'),
('VIGILANTE', 'Registra ingresos y salidas en portería'),

```

```
( 'USUARIO',      'Miembro de la institución con vehículos registrados');

INSERT INTO tipos_vehiculo (nombre) VALUES ('CARRO'), ('MOTO'), ('BICICLETA');

INSERT INTO zonas (nombre, descripcion) VALUES
('Zona A', 'Carros - entrada principal'),
('Zona B', 'Motos'),
('Zona C', 'Bicicletas');

INSERT INTO espacios (id_zona, id_tipo_vehiculo, codigo) VALUES
(1, 1, 'A-01'), (1, 1, 'A-02'), (1, 1, 'A-03'),
(2, 2, 'B-01'), (2, 2, 'B-02'),
(3, 3, 'C-01');

INSERT INTO usuarios (id_rol, documento, nombres, apellidos, correo, tipo_usuario, contrasena_hash)
VALUES
(1, '1000000001', 'Admin', 'Prueba', 'admin@institucion.edu.co', 'PERSONAL', '<hash>'),
(2, '1000000002', 'Vigilante', 'Prueba', 'vigilante@institucion.edu.co', 'PERSONAL', '<hash>'),
(3, '1000000003', 'Usuario', 'Uno', 'usuario1@institucion.edu.co', 'ESTUDIANTE', '<hash>'),
(3, '1000000004', 'Usuario', 'Dos', 'usuario2@institucion.edu.co', 'PROFESOR', '<hash>');

INSERT INTO vehiculos (id_usuario, id_tipo_vehiculo, placa, marca, modelo, color) VALUES
(3, 2, 'ABC12D', 'Marca X', '2022', 'Negro'),
(4, 1, 'XYZ123', 'Marca Y', '2020', 'Gris');
```

4. Consultas principales del sistema

4.1 1. Inicio de sesión: obtener usuario por correo (RF-01)

```
SELECT u.id_usuario, u.nombres, u.apellidos, u.contrasena_hash, u.estado, r.nombre AS rol
FROM usuarios u
JOIN roles r ON r.id_rol = u.id_rol
WHERE u.correo = ?;
```

4.2 2. Buscar vehículo por placa y validar autorización (RF-09, RF-10)

```
SELECT v.id_vehiculo, v.placa, v.estado, t.id_tipo_vehiculo, t.nombre AS tipo,
CONCAT(u.nombres, ' ', u.apellidos) AS propietario, u.estado AS estado_usuario,
(SELECT COUNT(*) FROM registros r
WHERE r.id_vehiculo = v.id_vehiculo AND r.fecha_hora_salida IS NULL) AS ingresos_abiertos
FROM vehiculos v
JOIN usuarios u ON u.id_usuario = v.id_usuario
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo
WHERE v.placa = ?;
```

4.3 3. Espacios disponibles para un tipo de vehículo (RF-09)

```
SELECT e.id_espacio, e.codigo, z.nombre AS zona
FROM espacios e
JOIN zonas z ON z.id_zona = e.id_zona
```

```
WHERE e.estado = 'DISPONIBLE' AND e.id_tipo_vehiculo = ?
ORDER BY z.nombre, e.codigo;
```

4.4 4. Registrar ingreso (transacción) (RF-09)

```
START TRANSACTION;
INSERT INTO registros (id_vehiculo, id_espacio, id_vigilante_ingreso)
VALUES (?, ?, ?);
UPDATE espacios SET estado = 'OCUPADO'
WHERE id_espacio = ? AND estado = 'DISPONIBLE';
COMMIT;
```

4.5 5. Registrar salida (transacción) (RF-11)

```
START TRANSACTION;
UPDATE registros
SET fecha_hora_salida = NOW(), id_vigilante_salida = ?
WHERE id_registro = ? AND fecha_hora_salida IS NULL;
UPDATE espacios SET estado = 'DISPONIBLE'
WHERE id_espacio = ?;
COMMIT;
```

4.6 6. Disponibilidad por zona y tipo (RF-12)

```
SELECT z.nombre AS zona, t.nombre AS tipo,
       SUM(e.estado = 'DISPONIBLE') AS disponibles,
       SUM(e.estado = 'OCUPADO') AS ocupados,
       SUM(e.estado = 'INHABILITADO') AS inhabilitados,
       COUNT(*) AS total
FROM espacios e
JOIN zonas z ON z.id_zona = e.id_zona
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = e.id_tipo_vehiculo
GROUP BY z.nombre, t.nombre
ORDER BY z.nombre;
```

4.7 7. Vehículos dentro del parqueadero (RF-13)

```
SELECT r.id_registro, v.placa, CONCAT(u.nombres, ' ', u.apellidos) AS propietario,
       e.codigo AS espacio, r.fecha_hora_ingreso
FROM registros r
JOIN vehiculos v ON v.id_vehiculo = r.id_vehiculo
JOIN usuarios u ON u.id_usuario = v.id_usuario
JOIN espacios e ON e.id_espacio = r.id_espacio
WHERE r.fecha_hora_salida IS NULL
ORDER BY r.fecha_hora_ingreso;
```

4.8 8. Historial por placa y rango de fechas (RF-14)

```
SELECT v.placa, e.codigo AS espacio, r.fecha_hora_ingreso, r.fecha_hora_salida,
       TIMESTAMPDIFF(MINUTE, r.fecha_hora_ingreso, r.fecha_hora_salida) AS minutos
FROM registros r
JOIN vehiculos v ON v.id_vehiculo = r.id_vehiculo
JOIN espacios e ON e.id_espacio = r.id_espacio
WHERE v.placa = ? AND r.fecha_hora_ingreso BETWEEN ? AND ?
ORDER BY r.fecha_hora_ingreso DESC;
```

4.9 9. Vehículos de un usuario (RF-16)

```
SELECT v.placa, t.nombre AS tipo, v.marca, v.modelo, v.color, v.estado
FROM vehiculos v
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo
WHERE v.id_usuario = ?;
```

4.10 10. Reporte de ingresos por día (RF-15)

```
SELECT DATE(r.fecha_hora_ingreso) AS dia, COUNT(*) AS ingresos
FROM registros r
WHERE r.fecha_hora_ingreso BETWEEN ? AND ?
GROUP BY DATE(r.fecha_hora_ingreso)
ORDER BY dia;
```